

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

Г.Э. Воцинская, Е.М. Лещенко

СТРУКТУРЫ И АЛГОРИТМЫ ОБРАБОТКИ ДАННЫХ

Часть 1

Учебно-методическое пособие

Воронеж
Издательский дом ВГУ
2019

Утверждено научно-методическим советом факультета прикладной математики, информатики и механики 20 марта 2019 г., протокол № 7

Рецензент – кандидат физико-математических наук, доцент О.Д. Горбенко

Учебно-методическое пособие подготовлено на кафедре программного обеспечения и администрирования информационных систем факультета прикладной математики, информатики и механики Воронежского государственного университета.

Рекомендовано студентам 2-го курса очного отделения для выполнения лабораторных занятий по предмету «Программирование на C#».

Для направления 02.03.03 – Математическое обеспечение и администрирование информационных систем

Содержание

Введение.....	4
Задание 1	18
Задание 2	21
Задание 3	28
Задание 4	32
Задание 5	37
Задание 6	44
Задание 7	50
Список литературы	53

Введение

Microsoft Visual Studio .NET – это интегрированная среда разработки (Integrated Development Environment (IDE)) для создания, документирования, запуска и отладки программ, написанных на языках .NET. Это мощный инструмент профессиональной разработки сложных приложений. Набор его функций чрезвычайно велик. Рассмотрим его некоторые элементы

Microsoft Visual Studio – это программная среда по разработке приложений для ОС Windows, как консольных, так и с графическим интерфейсом.

Функциональная структура среды включает в себя:

- редактор исходного кода, который включает множество дополнительных функций, как автодополнение IntelliSense, рефракторинг кода и т. д.;
- отладчик кода;
- редактор форм, предназначенный для упрощённого конструирования графических интерфейсов;
- веб-редактор;
- дизайнер классов;
- дизайнер схем баз данных.

Когда мы запускаем Visual Studio .NET, то видим следующее окно (рис.1):

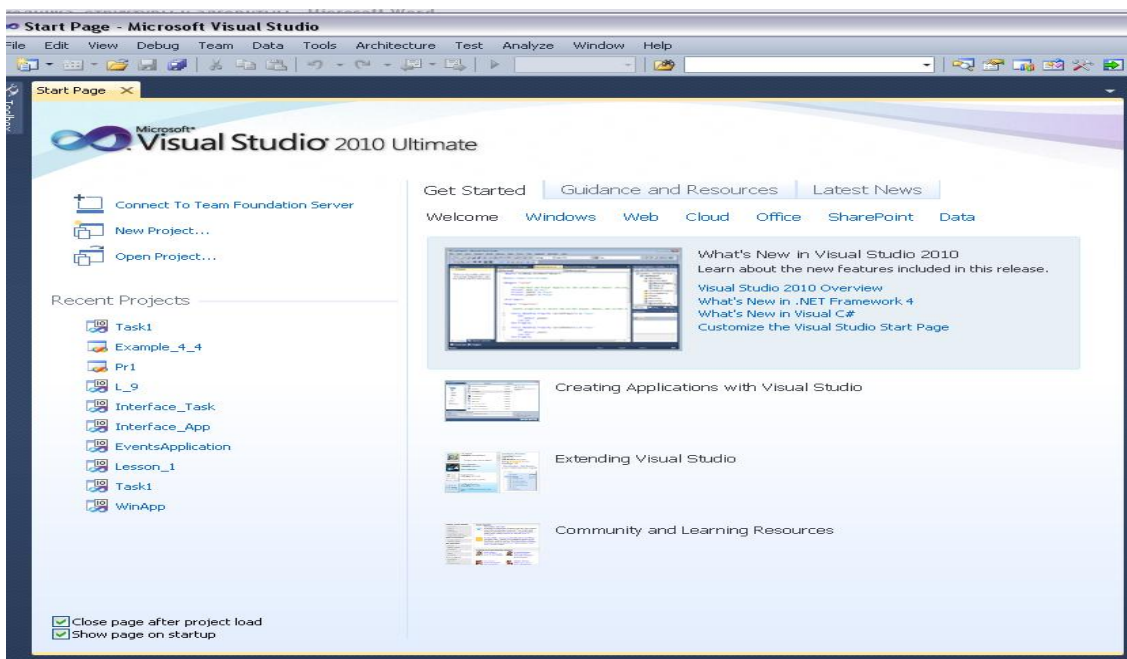


Рис. 1. Стартовая страница Visual Studio .NET

При создании нового проекта нужно выбрать New Project. Появится окно для выбора типа проекта (рис. 2). Для консольного проекта нужно выбрать Console Application, при создании Windows-приложения – Windows Forms Application. В поле Name задаем имя проекта и сохраняем его в папку, определяемую полем Location.

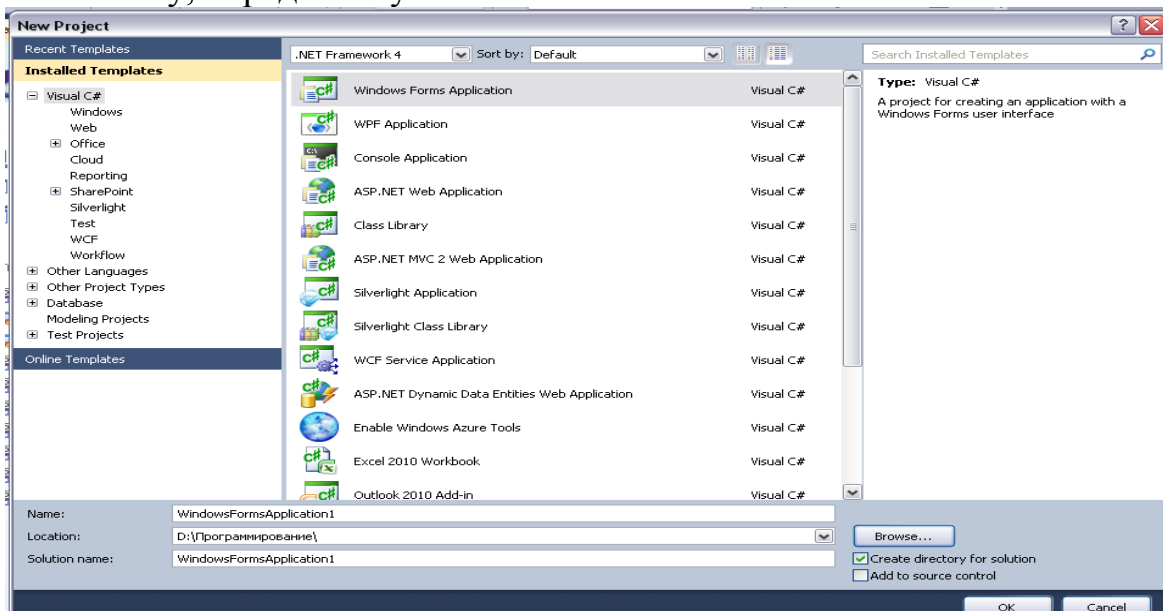


Рис.2. Окно для выбора типа проекта.

Создадим приложение на языке C# в Visual Studio. Для этого зайдите в студию. Затем выберите **FILE—>NEW—>Project** или нажмите комбинацию клавиш **Ctrl+Shift+N** и выберите **Console Application**(рис.3):

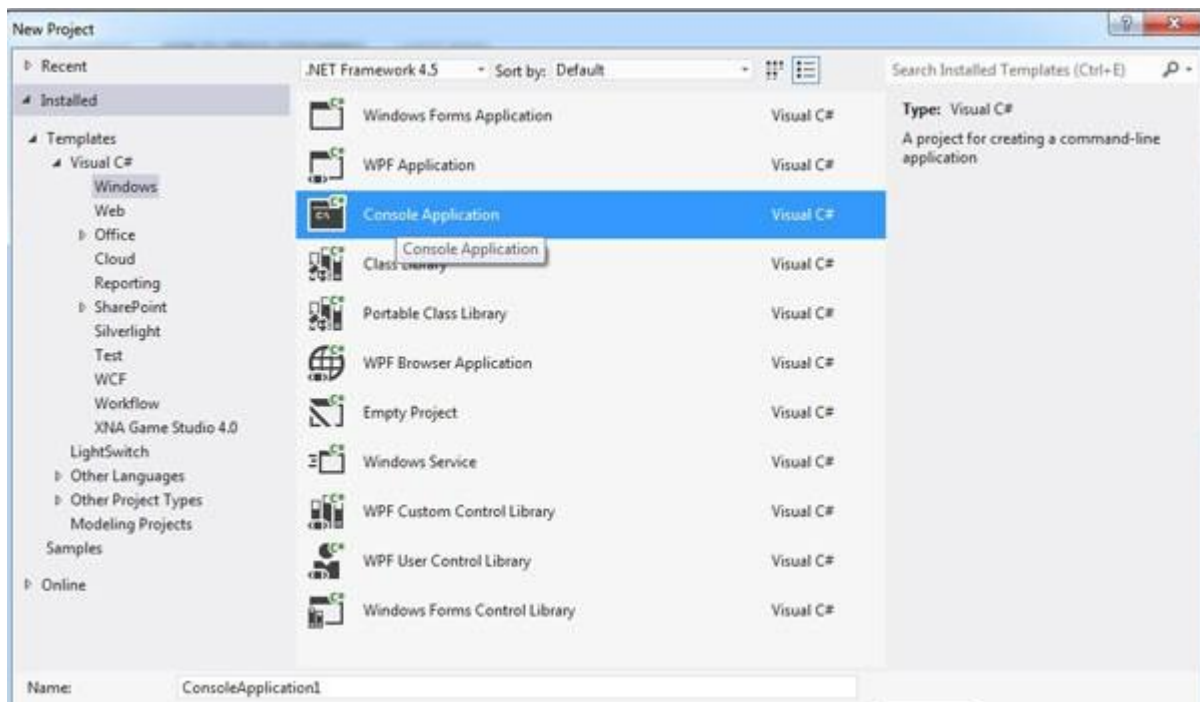


Рис. 3. Окно для создания консольного приложения.

Напишем первые строки на языке C# и запустим программу на выполнение, нажав на кнопку **"Start"** (или клавишу F5):

```
1 Console.WriteLine("Hello World!!!");  
2 Console.ReadLine();
```

Если у вас всё получилось правильно, будет выведено сообщение **"Hello World"**, и код должен выглядеть как на рисунке 4.

Кратко разберем основные моменты. Ключевое слово **using** используется для подключения библиотек к нашей программе. Библиотек существует большое количество. Если мы захотим работать с файлами, то необходимо будет подключать одну библиотеку, если с базами данных — другую.

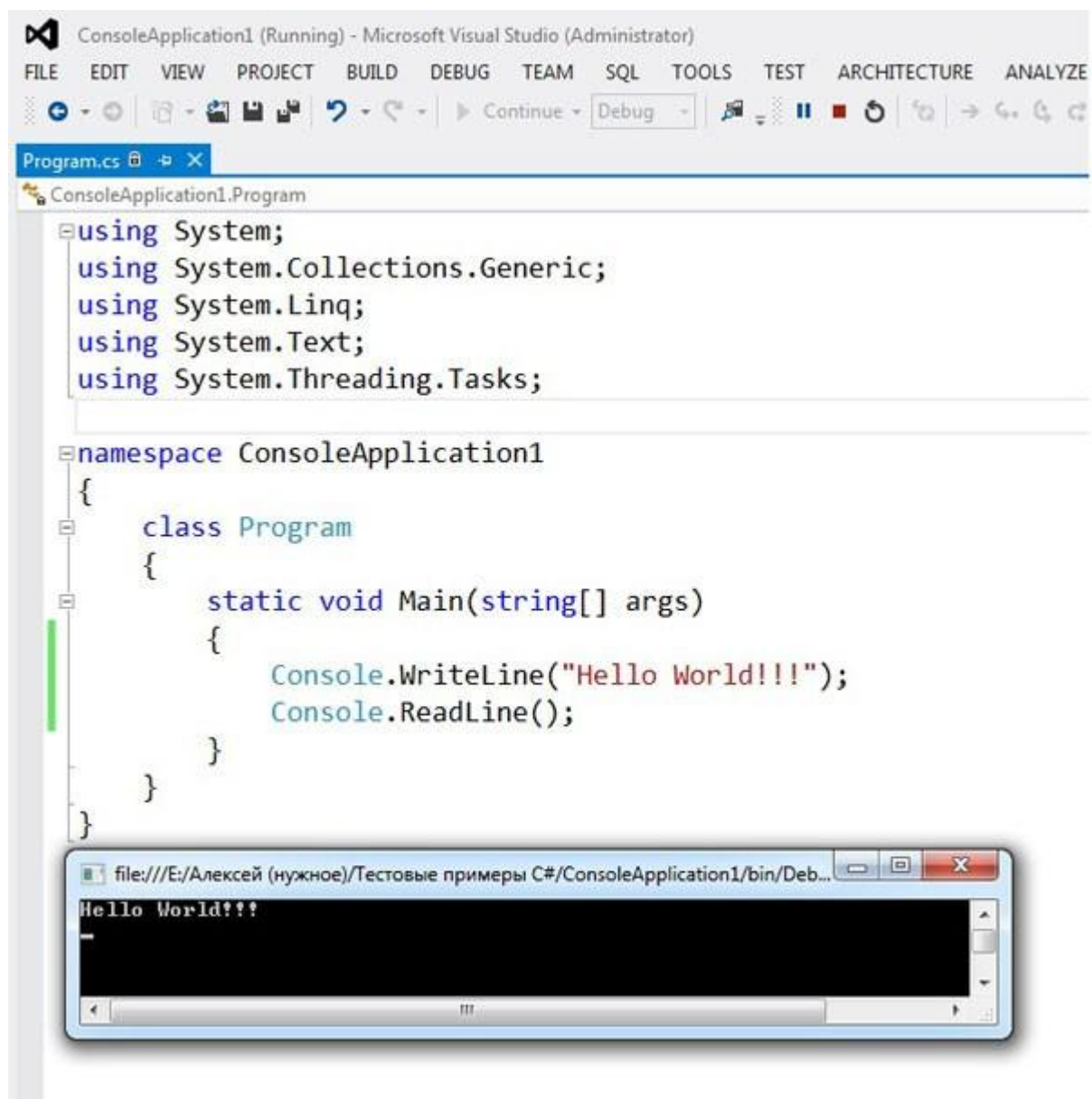


Рис.4. Пример программы.

namespace — это пространство имен. Обычно оно совпадает с названием приложения, хотя можно задать любое. По умолчанию среда Visual Studio назначает классу, определяющему метод **Main()** имя **Program**, хотя данное имя можно поменять. Метод **Main()** используется для обозначения точки входа в программу. Обратите внимание на ключевое слово **static** перед методом **Main()**. Это статический член, область действия которого охватывает уровень класса в целом, а не уровень объектов. Ключевое слово

void() означает, что возвращаемого значения не будет (т.е. в конце не будет стоять **return**).

Для вывода сообщения на экран используется класс **Console** (пространство имен **System**) со статическим методом **WriteLine()**, с помощью которого можно вывести строку текста на экран. Метод **Console.ReadLine()** нужен для того, чтобы мы видели результат работы программы до тех пор, пока не нажата клавиша "Enter".

С помощью специального окна **Properties** (Свойства) справа **Visual Studio** предоставляет нам удобный интерфейс для управления свойствами элемента.

Большинство этих свойств оказывает влияние на визуальное отображение формы. Рассмотрим некоторые из них:

- **Name**: устанавливает имя формы — точнее имя класса, который наследуется от класса **Form**.
- **BackColor**: указывает на фоновый цвет формы. Щелкнув на это свойство, можно выбрать тот цвет из списка предложенных цветов или цветовой палитры.
- **BackgroundImage**: указывает на фоновое изображение формы.
- **BackgroundImageLayout**: определяет, как изображение, заданное в свойстве **BackgroundImage**, будет располагаться на форме.
- **ControlBox**: указывает, отображается ли меню формы. В данном случае под меню понимается меню самого верхнего уровня, где находятся иконка приложения, заголовок формы, а также кнопки минимизации формы и крестик. Если данное свойство имеет значение **false**, то не будет видна ни иконка, ни крестик, с помощью которого обычно закрывается форма.

- **Cursor**: определяет тип курсора, который используется на форме.
- **Enabled**: если данное свойство имеет значение false, то форма не сможет получать ввод от пользователя, то есть нельзя будет нажать на кнопки, ввести текст в текстовые поля и т.д.
- **Font**: задает шрифт для всей формы и всех помещенных на нее элементов управления. Однако, задав у элементов формы свой шрифт, можно переопределить его.
- **ForeColor**: цвет шрифта на форме.
- **FormBorderStyle**: указывает, как будет отображаться граница формы и строка заголовка. Устанавливая данное свойство в None можно создавать внешний вид приложения произвольной формы.
- **HelpButton**: указывает, отображается ли кнопка справки формы.
- **Icon**: задает иконку формы.
- **Location**: определяет исходное положение формы по отношению к верхнему левому углу экрана, если для свойства StartPosition установлено значение Manual.
- **MaximizeBox**: указывает, будет ли доступна кнопка максимизации окна в заголовке формы.
- **MinimizeBox**: указывает, будет ли доступна кнопка минимизации окна.
- **MaximumSize**: задает максимальный размер формы.
- **MinimumSize**: задает минимальный размер формы.
- **Opacity**: задает прозрачность формы.

- **Size:** определяет начальный размер формы.
- **StartPosition:** указывает на начальную позицию, с которой форма появляется на экране.
- **Text:** определяет заголовок формы.
- **TopMost:** если данное свойство имеет значение true, то форма всегда будет находиться поверх других окон.
- **Visible:** видима ли форма. Если нужно скрыть форму от пользователя, то данное свойство устанавливается в false.
- **WindowState:** указывает, в каком состоянии форма будет находиться при запуске: в нормальном, максимизированном или минимизированном.

Программная настройка свойств

С помощью значений свойств в окне Свойства можно изменить по своему усмотрению внешний вид формы, но все то же самое можно сделать динамически в коде. Перейдем к коду, для этого нажмем правой кнопкой мыши на форме и выберем в появившемся контекстном меню View Code (Просмотр кода). В результате открывается файл кода *Form1.cs*. Изменим его следующим образом:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```

using System.Windows.Forms;
namespace HelloApp
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            Text = "Hello World!";
            this.BackColor = Color.Aquamarine;
            this.Width = 250;
            this.Height = 250;
        }
    }
}

```

В данном случае настроены несколько свойств отображения формы: заголовок, фоновый цвет, ширина и высота. При использовании конструктора формы надо учитывать, что весь остальной код должен идти после вызова метода `InitializeComponent()`, поэтому все установки свойств здесь расположены после этого метода.

Установка размеров формы

Для установки размеров формы можно использовать такие свойства как `Width/Height` или `Size`. `Width/Height` принимают числовые значения, как в вышеприведенном примере. При установке размеров через свойство `Size`, нам надо присвоить свойству объект типа `Size`:

```
this.Size = new Size(200,150);
```

Объект `Size` в свою очередь принимает в конструкторе числовые значения для установки ширины и высоты.

Начальное расположение формы

Начальное расположение формы устанавливается с помощью свойства `StartPosition`, которое может принимать одно из следующих значений:

- **Manual**: положение формы определяется свойством `Location`.
- **CenterScreen**: положение формы в центре экрана.
- **WindowsDefaultLocation**: позиция формы на экране задается системой Windows, а размер определяется свойством `Size`.
- **WindowsDefaultBounds**: начальная позиция и размер формы на экране задается системой Windows.
- **CenterParent**: положение формы устанавливается в центре родительского окна.

Все эти значения содержатся в перечислении `FormStartPosition`, поэтому, чтобы, например, установить форму в центре экрана, надо прописать так:

```
this.StartPosition = FormStartPosition.CenterScreen;
```

Фон и цвета формы

Чтобы установить цвет, как фона формы, так и шрифта, надо использовать цветовое значение, хранящееся в структуре `Color`:

```
this.BackColor = Color.Aquamarine;  
this.ForeColor = Color.Red;
```

Кроме того, можно в качестве фона задать изображение в свойстве `BackgroundImage`, выбрав его в окне свойств или в коде, указав путь к изображению:

```
this.BackgroundImage = Image.FromFile("C:\\Users\\Pictures\\3332.jpg");
```

Чтобы должным образом настроить нужное отображение фоновой картинки, надо использовать свойство `BackgroundImageLayout`, которое может принимать одно из следующих значений:

- **None:** Изображение помещается в верхнем левом углу формы и сохраняет свои первоначальные значения.
- **Tile:** Изображение располагается на форме в виде мозаики.
- **Center:** Изображение располагается по центру формы.
- **Stretch:** Изображение растягивается до размеров формы без сохранения пропорций.
- **Zoom:** Изображение растягивается до размеров формы с сохранением пропорций.

Например, расположим форму по центру экрана:

```
this.StartPosition = FormStartPosition.CenterScreen;
```

Для взаимодействия с пользователем в Windows Forms используется механизм событий. События в Windows Forms представляют стандартные события на C#, только применяемые к визуальным компонентам и подчиняются тем же правилам, что события в C#. Но создание обработчиков событий в Windows Forms все же имеет некоторые особенности.

Прежде всего в WinForms есть некоторый стандартный набор событий, который по большей части имеется у всех визуальных компонентов. Отдельные элементы добавляют свои события, но принципы работы с ними будут похожие. Чтобы посмотреть все события элемента, надо выбрать этот элемент в поле графического дизайнера и перейти к вкладке событий на панели форм. Например, события формы представлены на рисунке 5:

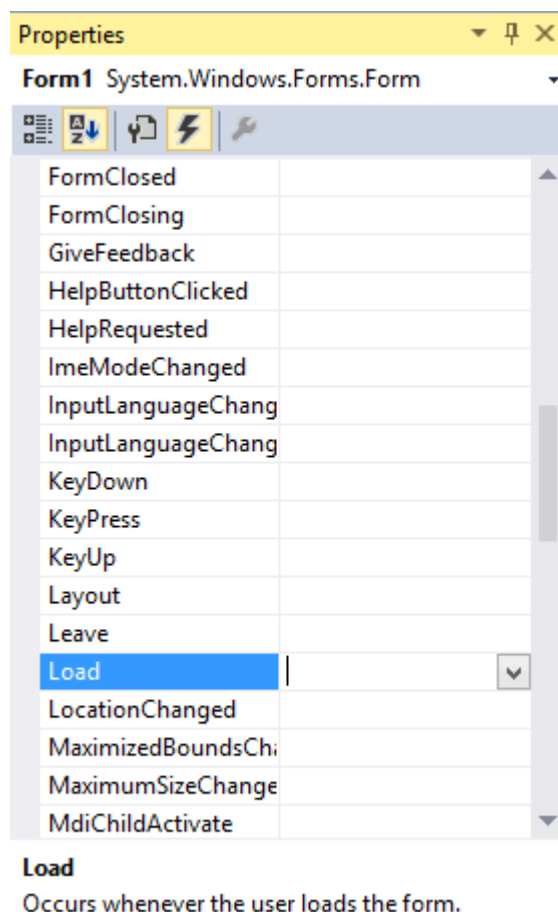


Рис.5. Страница свойств формы.

Чтобы добавить обработчик, можно просто два раза нажать по пустому полю рядом с названием события (рис. 6), и после этого Visual Studio автоматически сгенерирует обработчик события. Например, нажмем для создания обработчика для события Load ():

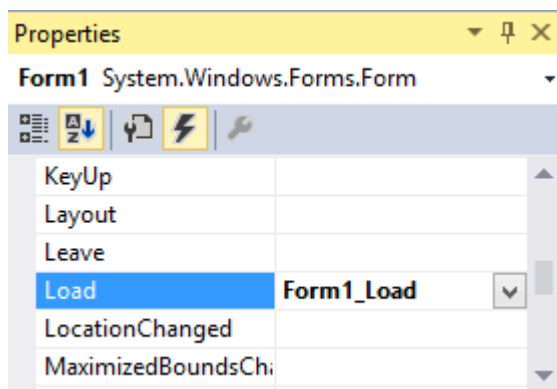


Рис. 6. Страница событий формы.

И в этом поле отобразится название метода обработчика события Load. По умолчанию он называется Form1_Load.

Если перейти в файл кода формы Form1.cs, то увидим автосгенерированный метод Form1_Load:

```
public partial class Form1 : Form
{
    public Form1 ()
    {
        InitializeComponent();
    }

    private void Form1_Load(object sender, EventArgs e)
    {
    }
}
```

И при каждой загрузке формы будет срабатывать код в обработчике Form1_Load.

Как правило, большинство обработчиков различных визуальных компонентов имеют два параметра: sender — объект, инициировавший событие, и аргумент, хранящий информацию о событии (в данном случае EventArgs e).

Но это только обработчик. Добавление же обработчика, созданного таким образом, производится в файле Form1.Designer.cs:

```
namespace HelloApp
{
    partial class Form1
    {
        private System.ComponentModel.IContainer
components = null;
        protected override void Dispose(bool disposing)
        {

```

```

        if (disposing && (components != null))
        {
            components.Dispose();
        }
        base.Dispose(disposing);
    }
    private void InitializeComponent()
    {
        this.SuspendLayout();

        this.AutoScaleDimensions =
new    System.Drawing.SizeF(6F, 13F);
        this.AutoScaleMode =
System.Windows.Forms.AutoScaleMode.Font;
        this.ClientSize = new
System.Drawing.Size(284, 261);
        this.Name = "Form1";
        // добавление обработчика
        this.Load += new
System.EventHandler(this.Form1_Load);
        this.ResumeLayout(false);
    }
}
}

```

Для добавления обработчика используется стандартный синтаксис C#:

```
this.Load += new System.EventHandler(this.Form1_Load);
```

Поэтому если нужно удалить созданный подобным образом обработчик, то надо не только удалить метод из кода формы в Form1.cs, но и удалить добавление обработчика в этом файле.

Однако можно добавлять обработчики событий и программно, например, в конструкторе формы:


```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace HelloApp
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            this.Load += LoadEvent;
        }
        private void Form1_Load(object sender, EventArgs e)
        {
        }

        private void LoadEvent(object sender, EventArgs e)
        {
            this.BackColor = Color.Yellow;
        }
    }
}

```

Кроме ранее созданного обработчика Form1_Load здесь также добавлен другой обработчик загрузки формы: this.Load += LoadEvent;, который устанавливает в качестве фона желтый цвет.

Задание 1

1. По каждому щелчку мыши изменить цвет формы и вывести в заголовок название цвета формы. Указание. Цвет формы определяется свойством `BackColor`. Значение цвета определяется перечислимым типом `Color`. Надпись заголовка формы определяется свойством формы `Text`.
2. По каждому щелчку мыши изменить размеры формы и вывести в заголовок формы размеры формы. Указание. Размеры формы определяются свойствами `Width` (ширина) и `Height` (высота). Надпись заголовка формы определяется свойством формы `Text`.
3. По каждому щелчку мыши изменить курсор мыши и вывести название курсора в заголовок формы. Указание. Курсор определяется свойством `Cursor`, а возможные значения — перечислимым типом `Cursors`. Надпись заголовка формы определяется свойством формы `Text`.
4. По каждому щелчку мыши вывести в заголовок формы координаты мыши. Указание. Координаты мыши определяются значением `Location` параметра `e` обработчика события `MouseClick`. Надпись заголовка формы определяется свойством формы `Text`.
5. По каждому щелчку мыши изменить стиль формы `FormBorderStyle` и вывести его название в заголовок формы. Указание. Стиль формы задается значениями перечислимого типа `FormBorderStyle`. Надпись заголовка формы определяется свойством формы `Text`.
6. По каждому щелчку мыши изменить иконку заголовка формы и вывести в заголовок ее название. Указание. Иконка формы определяется свойством `Icon`; новая иконка создается: `new Icon(“адрес файла с расширением ico”)`. Надпись заголовка формы определяется свойством формы `Text`. Название иконки — имя ее файла.
7. По каждому щелчку мыши изменить координаты формы и вывести в заголовок координаты формы. Указание. Координаты формы опреде-

ляются свойством Location. Заголовок формы задается свойством Text.

8. По каждому нажатию на кнопку изменить цвет формы и вывести на кнопку название цвета формы. Указание. Цвет формы определяется свойством BackColor. Значение цвета определяется перечислимым типом Color.
9. По каждому нажатию на кнопку изменить размеры кнопки и вывести на кнопку размеры кнопки. Указание. Размеры кнопки определяются свойством кнопки Size. Надпись на кнопке задается свойством кнопки Text.
10. По каждому нажатию на кнопку изменить размеры формы и вывести на кнопку размеры формы. Указание. Размеры формы определяются свойствами Width (ширина) и Height (высота). Надпись на кнопке задается свойством кнопки Text.
11. По каждому нажатию на кнопку изменить размеры формы вывести в заголовок формы размеры формы. Указание. Размеры формы определяются свойствами Width (ширина) и Height (высота). Надпись на форме задается свойством формы Text.
12. По каждому нажатию на кнопку изменить координаты кнопки и вывести на кнопку ее координаты. Указание. Координаты кнопки определяются свойством Location. Надпись на кнопке задается свойством кнопки Text.
13. По каждому нажатию на кнопку изменить цвет кнопки и вывести на кнопку название цвета кнопки. Цвет кнопки определяется свойством кнопки BackColor. Значение цвета определяется перечислимым типом Color. Надпись на кнопке задается свойством кнопки Text.
14. По каждому нажатию на кнопку изменить цвет шрифта кнопки и вывести на кнопку название цвета шрифта кнопки. Указание. Цвет

шрифта кнопки определяется свойством кнопки `ForeColor`. Название шрифта задается свойством кнопки `Font.Name`. Надпись на кнопке задается свойством кнопки `Text`.

15. По нажатию на кнопку изменить `BackgroundImage` кнопки и вывести в заголовок формы название `BackgroundImage`. Указание. Значение свойства кнопки `BackgroundImage` создается: `new Bitmap("адрес файла с расширением ico ")`; Надпись на форме задается свойством формы `Text`.
16. По нажатию на кнопку изменить курсор и вывести в заголовок формы название курсора. Указание. Курсор определяется свойством `Cursor`, а возможные значения — перечислимый тип `Cursors`. Надпись на форме задается свойством формы `Text`.
17. По нажатию на кнопку изменить `FlatStyle` кнопки и вывести на кнопку значение `FlatStyle`. Указание. Свойство кнопки `FlatStyle` определяется значениями перечислимого типа `FlatStyle`. Надпись на кнопке задается свойством кнопки `Text`.
18. По нажатию на первую кнопку изменить ее цвет и вывести на кнопку название цвета, при нажатии на вторую кнопку сделать ее цвет таким же, как у первой. Указание. Цвет кнопки определяется свойством кнопки `BackColor`. Значение цвета определяется перечислимый типом `Color`. Надпись на кнопке задается свойством кнопки `Text`.
19. По нажатию на кнопку изменить шрифт кнопки и вывести на кнопку его название. Указание. Надпись на кнопке задается свойством кнопки `Text`. Значение шрифта задается `new Font("название шрифта", размер шрифта, Стиль шрифта)`. Стиль шрифта задается значением перечислимого типа `FontStyle`.

20. По нажатию на кнопку изменить положение формы и вывести в заголовков координаты формы. Указание. Координаты формы определяются свойством Location. Заголовок формы задается свойством Text.

Задание 2

Реализовать консольное приложение для решения следующей задачи.

Вычислить значения функции, заданной с помощью ряда Тейлора, на интервале от X_0 до X_n с шагом h с заданной точностью ϵ . Результаты вывести в виде таблицы.

x									
f(x) вы- числ.									
f(x) по формуле									

x									
f(x) вы- числ									
f(x) по формуле									

1. $\frac{\sin(x)}{x} = 1 - \frac{x^2}{3!} + \frac{x^4}{5!} - \frac{x^6}{7!} + \dots$ (R= ∞).

2. $e^{-x^2} = 1 - \frac{x^2}{1!} + \frac{x^4}{2!} - \dots + (-1)^N * \frac{x^{2*N}}{N!}$ (R= ∞).

3. $\ln(x + \sqrt{x^2 + 1}) = x - \frac{1}{2} * \frac{x^3}{3} + \frac{1*3}{2*4} * \frac{x^5}{5} - \frac{1*3*5}{2*4*6} * \frac{x^7}{7} + \dots$ (R=1)

4. $\arctg(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \frac{x^9}{9} - \dots$ (R=1).

$$5. \quad \arcsin(x) = x + \frac{1}{2} * \frac{x^3}{3} + \frac{1*3}{2*4} * \frac{x^5}{5} + \frac{1*3*5}{2*4*6} * \frac{x^7}{7} + \dots \quad (R=1).$$

$$6. \quad \frac{1}{\sqrt{1-x^2}} = 1 + \frac{1}{2} * x^2 + \frac{1*3}{2*4} * x^4 + \frac{1*3*5}{2*4*6} * x^6 + \dots \quad (R=1).$$

$$7. \quad \frac{1}{\sqrt{1+x}} = 1 - \frac{1}{2} * x + \frac{1*3}{2*4} * x^2 - \frac{1*3*5}{2*4*6} * x^3 + \dots \quad (R=1).$$

$$8. \quad \sqrt{1+x} = 1 + \frac{1}{2} * x - \frac{1}{2*4} * x^2 + \frac{1*3}{2*4*6} * x^3 - \dots \quad (R=1).$$

$$9. \quad \frac{1}{(1+x)^3} = 1 - \frac{2*3}{2} * x + \frac{3*4}{2} * x^2 - \frac{4*5}{2} * x^3 + \dots \quad (R=1)$$

$$10. \quad \frac{1}{(1+x)^2} = 1 - 2 * x + 3 * x^2 - 4 * x^3 + 5 * x^4 - \dots \quad (R=1).$$

$$11. \quad \frac{1}{1+x} = 1 - x + x^2 - x^3 + x^4 - \dots \quad (R=1).$$

$$12. \quad \ln \frac{1+x}{1-x} = 2 * \left(x + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \dots \right) \quad (R=1).$$

$$13. \quad \ln(1-x) = -\frac{x}{1} - \frac{x^2}{2} - \frac{x^3}{3} - \frac{x^4}{4} - \dots \quad (R=1).$$

$$14. \quad \ln(1+x) = \frac{x}{1} - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots \quad (R=1).$$

$$15. \quad ch(x) = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \dots \quad (R=\infty).$$

$$16. \quad sh(x) = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \frac{x^7}{7!} + \dots \quad (R=\infty).$$

$$17. \quad e^{-x} = 1 - \frac{x}{1!} + \frac{x^2}{2!} - \dots + (-1)^N * \frac{x^N}{N!} \quad (R=\infty).$$

$$18. \quad e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \dots + \frac{x^N}{N!} \quad (R=\infty).$$

$$19. \quad \cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots \quad (R=\infty).$$

$$20. \quad \sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \quad (R=\infty).$$

Консольный ввод-вывод

Главные методы класса `Console` – это методы `Read()`, `ReadLine()`, `Write()`, `WriteLine()`.

Метод `static int Read()` используется для считывания одного символа. Результат возвращает как значение типа `int`, которое должно быть приведено к типу `char`.

Метод `static string ReadLine()` считывает строку символов. При вводе числовых значений они должны быть переведены с помощью следующих функций(табл.1).

Таблица1. Методы преобразования строки

<i>Структура</i>	<i>Метод преобразования</i>
<code>Decimal</code>	<code>static decimal Parse(string str)</code>
<code>Double</code>	<code>static double Parse(string str)</code>
<code>Single</code>	<code>static float Parse(string str)</code>
<code>Int64</code>	<code>static long Parse(string str)</code>
<code>Int32</code>	<code>static int Parse(string str)</code>
<code>Int16</code>	<code>static short Parse(string str)</code>
<code>UInt64</code>	<code>static ulong Parse(string str)</code>
<code>UInt32</code>	<code>static uint Parse(string str)</code>
<code>UInt16</code>	<code>static ushort Parse(string str)</code>
<code>Byte</code>	<code>static byte Parse(string str)</code>

Вывод информации в окно приложения обеспечивают методы `Console.Write()` и `Console.WriteLine()`.

`WriteLine()` отличается тем, что завершает свою работу обязательным выводом Escape-последовательности `line feed/carriage return`.

```
Console.WriteLine("The Captain is on the board!"); // Вывод строки.  
Console.WriteLine(314); //Символьное представление целочисленного
```

//значения.

**Console.WriteLine(3.14); //Символьное представление значения типа
//float.**

Escape-последовательность `\n` внутри строки означает переход на новую строку.

Escape-последовательность `\t` внутри строки означает табуляцию.

Методы с одним параметром достаточно просты. На стадии компиляции при выяснении типа выводимого значения подбирается соответствующий вариант перегруженной функции вывода.

При выводе значения определяется его тип, производится соответствующее стандартное преобразование к символьному виду, которое в виде последовательности символов, соответствующей существующей в операционной системе настройке языковых параметров, представления чисел, времени и дат, и выводится в окно представления.

При использовании метода `WriteLine()` с несколькими параметрами первый *обязательно строковый* параметр используется как управляющий шаблон для представления выводимой информации. Значения следующих за строковым параметром выражений будут выводиться в окно представления лишь в том случае, если первый параметр-строка будет явно указывать места расположения выводимых значений, соответствующих этим параметрам. Явное указание обеспечивается маркерами выводимых значений, которые в самом простом случае представляют собой заключенные в фигурные скобки целочисленные литералы (например, `{3}`). Таким образом, оператор `Console.WriteLine("The sum of {0} and {1} is {2}",314,3.14,314+3.14);`

обеспечивает вывод следующей строки:

The sum of 314 and 3.14 is 317.14

Помимо индекса параметра, маркер выводимого значения может содержать дополнительные сведения относительно формата представления

выводимой информации. Выводимые значения преобразуются к символьному представлению, которое, в свою очередь, при выводе в окно приложения может быть дополнительно преобразовано в соответствии с определенным "сценарием преобразования". Вся необходимая для дополнительного форматирования информация размещается непосредственно в маркерах и отделяется запятой от индекса маркера.

WriteLine("строка_форматирования", arg0, arg1,...,argN)

строка_форматирования::={номер_аргумента, ширина:формат}

При наличии элемента *ширина* выводимые данные дополняются пробелами, которые гарантируют, что поле, занимаемое выводимым значением, будет иметь минимальную ширину. Если значение *ширина* положительно, выводимые данные выравниваются по правому краю, если отрицательно – то по левому.

Кроме того, в маркерах вывода могут также размещаться дополнительные строки форматирования (FormatString). При этом маркер приобретает достаточно сложную структуру, внешний вид которой в общем случае можно представить следующим образом (M – значение индекса, N – область позиционирования):

{M, N:FormatString},

либо

{M:FormatString},

если не указывается значение области позиционирования.

Непосредственно за символом форматирования может быть расположена целочисленная ограничительная константа, которая, в зависимости от типа выводимого значения, может определять количество выводимых знаков после точки либо общее количество выводимых символов. При этом дробная часть действительных значений округляется или дополняется нулями справа. При выводе целочисленных значений ограничительная константа игнорируется, если количество выводимых символов превышает ее

значение. В противном случае выводимое значение слева дополняется нулями.

Следующие примеры иллюстрируют варианты применения маркеров со строками форматирования:

```
Console.WriteLine("Integer fotmating - {0:D3},{1:D5}",12345, 12);
Console.WriteLine("Currency formatting - {0:C},{1:C5}", 99.9, 999.9);
Console.WriteLine("Exponential formatting - {0:E}", 1234.5);
Console.WriteLine("Fixed Point formatting - {0:F3}", 1234.56789);
Console.WriteLine("General formatting - {0:G}", 1234.56789);
Console.WriteLine("Number formatting - {0:N}", 1234567.89);
Console.WriteLine("Hexadecimal formatting - {0:X7}",12345);//Integers
//only!
ConsoleWrteline("При делении 10/3 получаем: {0:#.##}",10.0/3.0);
ConsoleWrteline("{0:###,###.##}",123456.56);
```

В результате выполнения этих операторов в окно консольного приложения будут выведены следующие строки:

```
Integer fotmating – 12345,00012
Currency formatting – $99.90,$999.90000
Exponential formatting – 1.234500E+003
Fixed Point formatting – 1234.568
General formatting – 1234.56789
Number formatting – 1,234,567.89
Hexadecimal formatting – 0003039
При делении 10/3 получаем: 3.33
123,456.56
```

Пример программы

Найти среднее арифметическое чисел, введенных пользователем.

```
using System;
using System.IO;

class AvgNums {
    public static void Main() {
        string str;
        int n;
        double sum = 0.0;
        double avg, t;

        Console.Write("Введите количество чисел: ");
        str = Console.ReadLine();
        try {
            n = Int32.Parse(str);
        }
        catch(FormatException exc) {
            Console.WriteLine(exc.Message);
            n = 0;
        }
        catch(OverflowException exc) {
            Console.WriteLine(exc.Message);
            n = 0;
        }

        Console.WriteLine("Введите " + n + " чисел.");
        for(int i=0; i < n ; i++) {
            Console.Write(": ");
            str = Console.ReadLine();
```

```

    try {
        t = Double.Parse(str);
    } catch (FormatException exc) {
        Console.WriteLine(exc.Message);
        t = 0.0;
    }
    catch (OverflowException exc) {
        Console.WriteLine(exc.Message);
        t = 0;
    }
    sum += t;
}
avg = sum / n;
Console.WriteLine("Среднее =" + avg);
}
}

```

Задание 3

Указание. Реализовать задачу как консольный проект. Размер массива задается вводом.

1. Даны действительные числа $x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n, r_1, r_2, \dots, r_n$, определяющие соответственно координаты центров и радиусы n кругов. Выяснить, есть ли круг, пересекающийся со всеми остальными кругами.
2. Даны действительные числа $x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n, r_1, r_2, \dots, r_n$, определяющие соответственно координаты центров и радиусы n кругов. Выяснить, есть ли круг, не пересекающийся ни с одним другим.

3. Даны действительные числа $a_1, a_2, \dots, a_{2n}$. Эти точки определяют n интервалов числовой оси $(a_1, a_2), (a_3, a_4), \dots, (a_{2n-1}, a_{2n})$. Определить, есть ли интервал, который является частью всех остальных. Если да, то указать концы этого интервала.
4. Даны действительные числа $a_1, a_2, \dots, a_{2n}$. Эти точки определяют n интервалов числовой оси $(a_1, a_2), (a_3, a_4), \dots, (a_{2n-1}, a_{2n})$. Является ли интервалом объединение этих интервалов? Если да, то указать концы этого интервала.
5. Даны действительные числа $a_1, a_2, \dots, a_{2n}$. Эти точки определяют n интервалов числовой $(a_1, a_2), (a_3, a_4), \dots, (a_{2n-1}, a_{2n})$. Имеются ли точки числовой оси, принадлежащие, по крайней мере, трем каким-нибудь из данных интервалов. Если да, то указать какую-нибудь из этих точек.
6. Даны координаты центров n окружностей и их радиусы. Определить число пересекающихся окружностей.
7. Все отрицательные элементы массива X перенести в его начало, а все остальные – в конец, сохраняя исходное взаимное расположение, как среди отрицательных, так и среди остальных элементов. Дополнительный массив не использовать.
8. Прямая на плоскости может быть задана уравнением $ax + by = c$, где a, b одновременно не равны нулю, a, b, c – целые. Пусть даны коэффициенты n прямых $a_1, b_1, c_1, a_2, b_2, c_2, \dots, a_n, b_n, c_n$. Определить, имеются ли среди этих прямых три прямые, пересекающиеся в одной точке.
9. Прямая на плоскости может быть задана уравнением $ax + by = c$, где a, b одновременно не равны нулю, a, b, c – целые. Пусть даны коэффициенты n прямых $a_1, b_1, c_1, a_2, b_2, c_2, \dots, a_n, b_n, c_n$. Определить, имеются ли среди этих прямых параллельные.

10. Дан массив размера n . Определить количество его промежутков монотонности (то есть участков, на которых его элементы возрастают или убывают).
11. Дан целочисленный массив размера n . Определить максимальное количество его одинаковых элементов.
12. Дан целочисленный массив размера n . Если он является перестановкой, то есть содержит все числа от 1 до n , то вывести 0, в противном случае вывести номер первого недопустимого элемента.
13. Дано число k и целочисленный массив размера n . Поменять местами первую и k -ю серии равных элементов массива. Если серий в массиве меньше k , то вывести массив без изменений.
14. Дано число k и целочисленный массив размера n . Поменять местами последнюю и k -ю серии положительных элементов массива. Если серий в массиве меньше k , то вывести массив без изменений.
15. Дан массив A из n точек. Найти наибольший периметр треугольника, вершины которого принадлежат различным точкам массива A , и сами эти точки (точки выводятся в том же порядке, в котором они перечислены при задании массива A).
16. Дан массив A из n точек. Найти наименьший периметр треугольника, вершины которого принадлежат различным точкам массива A , и сами эти точки (точки выводятся в том же порядке, в котором они перечислены при задании массива A).
17. Дан массив из n точек на прямой. Найти такую точку из данного массива, сумма расстояний от которой до остальных его точек минимальна, и саму эту сумму.
18. Дано число k и целочисленный массив размера n . Назовем *серией* группу подряд идущих одинаковых элементов, а длиной серии — ко-

личество этих элементов (длина серии может быть равна 1). Удалить из массива все серии, длина которых меньше k .

19. Дано число k и целочисленный массив размера n . Назовем *серией* группу подряд идущих одинаковых элементов, а длиной серии — количество этих элементов (длина серии может быть равна 1). Удалить из массива все серии, длина которых равна k .

20. Дано число k и целочисленный массив размера n . Назовем *серией* группу подряд идущих одинаковых элементов, а длиной серии — количество этих элементов (длина серии может быть равна 1). Удалить из массива все серии, длина которых больше k .

Пояснения.

Описание массива:

```
тип[] имя;  
тип[] имя = new тип [ размерность ];
```

Пример.

Определить сумму и количество отрицательных элементов.

```
using System;  
namespace ConsoleApplication1  
{  
    class Class1  
    {  
        static void Main()  
        {  
            const int n = 6;
```

```

int[] a = new int[n] { 3, 12, 5, -9, 8, -4 };

Console.WriteLine( "Исходный массив:" );
for ( int i = 0; i < n; ++i )
    Console.Write( "\t" + a[i] );
Console.WriteLine();

long sum = 0;      // сумма отрицательных элементов
int  num = 0;      // количество отрицательных элементов
for ( int i = 0; i < n; ++i )
    if ( a[i] < 0 )
    {
        sum += a[i];
        ++num;
    }

Console.WriteLine( "Сумма отрицательных = " + sum );
Console.WriteLine( "Кол-во отрицательных = " + num );

    }
}
}

```

Задание 4

Цель задания – изучение механизма передачи данных функциями.

Указание. Реализовать задачу как консольный проект. Использовать статические массивы. Функция, реализующая поставленную задачу, должна иметь параметры, передаваемые различными способами, должна возвращать результат – массив, остальные данные передаются и возвраща-

ются параметрами: размер исходного массива – по значению, исходный массив – по ссылке **ref**, размер массива – результата – по ссылке **out**.

1. В данной матрице размером $n * n$ для каждой строки найти все отрицательные, которые повторяются.
2. В данной матрице размером $n * n$ для каждой строки найти все простые числа.
3. В данной матрице размером $n * n$ для каждой строки найти все числа Фибоначчи.
4. В данной матрице размером $n * n$ для каждой строки найти все симметричные числа (симметричное, например, 121, 4554).
5. В данной матрице размером $n * n$ для каждой строки найти все числа, которые являются степенью 2.
6. В данной матрице размером $n * n$ для каждой строки найти серию наибольшей длины из положительных.
7. В данной матрице размером $n * n$ для каждой строки найти возрастающую серию наибольшей длины.
8. В данной матрице размером $n * n$ для каждой строки найти все значения между первым минимальным и последним максимальным.
9. В данной матрице размером $n * n$ для каждой строки найти позиции локальных максимумов.
10. В данной матрице размером $n * n$ для каждой строки найти все позиции максимумов.
11. В данной матрице размером $n * n$ для каждого столбца найти все положительные, которые не повторяются.

12. В данной матрице размером $n * n$ для каждого столбца найти все непростые числа.
13. В данной матрице размером $n * n$ для каждого столбца найти все значения, не являющиеся числами Фибоначчи.
14. В данной матрице размером $n * n$ для каждого столбца найти все не-симметричные числа (симметричное, например, 121, 4554).
15. В данной матрице размером $n * n$ для каждого столбца найти все числа, которые не являются степенью 2.
16. В данной матрице размером $n * n$ для каждого столбца найти серию наибольшей длины из отрицательных.
17. В данной матрице размером $n * n$ для каждого столбца найти убывающую серию наибольшей длины.
18. В данной матрице размером $n * n$ для каждого столбца найти все значения между последним минимальным и первым максимальным.
19. В данной матрице размером $n * n$ для каждого столбца найти позиции локальных минимумов.
20. В данной матрице размером $n * n$ для каждого столбца найти все позиции минимумов.

Пример.

В матрице $n*n$ для каждой строки найти множители максимального значения

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;
```

```

namespace OutArray
{

    class Factor
    {
        /* Метод возвращает массив, содержащий множители параметра
           num. После выполнения метода out-параметр numfactors
будет содержать количество найденных множителей. */
        static public int[] findfactors(int num, out int numfactors)
        {
            int[] facts = new int[10]; // Размер 10 взят
                                     // произвольно

            int i, j;

            // Находим множители и помещаем их в массив facts.
            for (i = 2, j = 0; i < num / 2 + 1; i++)
                if ((num % i) == 0)
                {
                    facts[j] = i;
                    j++;
                }
            facts[j] = num;
            j++;
            numfactors = j;
            return facts;

        }

        // метод возвращает массив множителей максимального значения
        // строки n_str матрицы in_arr рамера size * size.
        // Размер массива - результата – numfactors.

```

```

static int[] test(int size, int n_str, ref int[,] in_arr,
                  out int numfactors)
{
    int[] facts = new int[10]; // Размер 10 взят
                                // произвольно

    int i;
    int max = in_arr[n_str,0];
    for (i = 1; i < size; i++)
        if (in_arr[n_str,i] > max)
            max = in_arr[n_str,i];
    return findfactors(max, out numfactors);
}

public static void Main()
{
    const int n = 5;
    int numfactors;
    int [,] in_arr=new int[n,n];

    int[] factors;
    Random rnd=new Random();

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            in_arr[i, j] = rnd.Next(100);

    Console.WriteLine("произвольно заданная матрица:");
    for (int i = 0; i < in_arr.GetLength(0); i++)
    {
        for (int j = 0; j < in_arr.GetLength(1); j++)
            Console.Write("\t" + in_arr[i, j]);
        Console.WriteLine();
    }
}

```

```

        for (int j = 0; j < n; j++)
        {

            factors = test(n,j, ref in_arr,out numfactors);
            Console.WriteLine("Множители max {0} строки: ",j);
            for (int i = 0; i < numfactors; i++)
                Console.Write(factors[i] + " ");
            Console.WriteLine();

        }
    }
}

```

Задание 5

Реализовать GUI – приложение.

Требования.

1. Функциональность должна быть реализована в виде меню.
2. Программа должна позволять:
 - a. создавать новый файл;
 - b. открывать существующий;
 - c. сохранять его;
 - d. сохранять под другим именем;
 - e. обрабатывать открытый файл;
 - f. сохранять результат обработки (в случае необходимости);
 - g. очищать результат обработки.

Задания

1. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать на его основе пять файлов. В первый файл должны войти слова длины пять, во второй — слова длины четыре и т. д. Если слов определенной длины не будет найдено, соответствующий файл должен быть пуст.
2. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать файл целых чисел, в котором каждой строке исходного файла соответствует в выходном файле число, равное количеству слов в ней. Пустой строке или строке, состоящей из одних пробелов, соответствует число ноль.
3. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать новый файл, каждая строка которого содержит слово исходного файла, имеющее номер $(n+1)div2$, где n — количество слов в соответствующей строке исходного файла.
4. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать файл, каждая строка которого должна содержать последнее из слов соответствующей строки исходного файла, в котором наибольшее число различных букв.
5. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Перенести в новый файл только те строки, которые предшествуют строкам, начинающимся с заданного слова.
6. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Переписать его содержимое в новый файл, сохраняя строчную структуру и удаляя пустые строки. Строка, состоящая только из одних пробелов, тоже считается пустой.
7. Дан текстовый файл. Проверить, упорядочены ли строки файла по количеству символов в них.

8. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать на основе новый файл, состоящий только из тех строк первого файла, которые содержат заданное слово.
9. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать на его основе новый файл, состоящий только из тех строк первого файла, которые не содержат заданное слово.
10. Дан текстовый файл. Известно, что в нем записаны целые числа. Записать в один файл все четные числа исходного файла, а в другой — все нечетные.
11. Даны два текстовых файла. Написать программу, которая сравнивала бы их на совпадение и выводила бы в качестве результата номер строки и номер символа, где встретилось первое отличие.
12. Дан текстовый файл. Получить новый файл, образованный из исходного заменой всех больших латинских и русских букв на малые.
13. Дан текстовый файл. Перенести в новый файл те строки исходного файла, которые начинаются и заканчиваются одной и той же буквой (вне зависимости от регистра).
14. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Реализовать программу, позволяющую на основе исходного файла формировать новый файл, где каждое слово расположено на отдельной строке. Переход к новой строке в исходном файле соответствует пустой строке в новом файле.
15. Задан текстовый файл, состоящий из слов, расположенных на отдельной строке. Сформировать новый файл, который будет состоять из слов исходного файла, разделенных одним пробелом. Пустая строка исходного файла будет соответствовать переходу на новую строку в создаваемом файле.
16. Дан текстовый файл, слова содержащий слова, разделенные одним или

несколькими пробелами. Создать новый файл, содержащий строки с наибольшим числом различных слов.

17. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Сформировать новый файл, каждая строка которого должна содержать слово наибольшей длины из соответствующей строки исходного файла. Если таких слов несколько, то они должны войти все.

18. Дан текстовый файл, слова в котором разделены одним или несколькими пробелами. Сформировать новый файл, удалив из исходного строки, заканчивающиеся заданным словом.

19. Дан текстовый файл, слова в котором разделены одним или несколькими пробелами. Сформировать три новых файла. Первый файл должен содержать только те строки исходного файла, количество слов в которых больше заданного значения k . Второй файл должен содержать те строки исходного файла, количество слов в которых меньше заданного значения k , а третий - только строки, содержащие ровно k слов.

20. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать два выходных файла. В один переписать из каждой строки первые k слов, а в другой – оставшиеся. Если в строке меньше чем k слов, то во втором файле соответствующая строка должна быть пустой.

21. Дан текстовый файл. Сформировать на его основе новый файл, удалив из него те строки исходного файла, которые содержат наибольшее число различных символов, отличных от разделителя. Разделителем считается пробел, запятая, точка, двоеточие, точка с запятой, восклицательный и вопросительный знаки.

22. Дан текстовый файл, слова в котором разделены одним или несколькими пробелами. Перенести в новый файл только те строки, все слова кото-

рых состоят из повторяющихся букв.

23. Дан текстовый файл, слова в котором разделены одним или несколькими пробелами. Перенести в новый файл только те строки, в которых нет слов, состоящих из повторяющихся букв.

24. Дан текстовый файл, содержащий слова, разделенные одним или несколькими пробелами. Создать новый файл, исключив из строк исходного файла все слова, которые содержат хотя бы одну букву из заданного набора.

Пример

Дан текстовый файл. Переписать его содержимое в другой файл, каждая строка которого имеет следующую структуру:
номер строки; строка; длина строки.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.IO;
namespace Задание_5
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

```

        private void openToolStripMenuItem_Click(object sender,
                                                    EventArgs e)
        {
            if (fileName == "")
            {
if(openFileDialog1.ShowDialog()==
System.Windows.Forms.DialogResult.OK)
            {
                f_In = new StreamReader(openFileDialog1.FileName);
                fileName = openFileDialog1.FileName;
                textBox1.Text = f_In.ReadToEnd();
            }
        }
    }

    private void saveToolStripMenuItem_Click(object sender,
        EventArgs e)
    {
        if (fileName == "")
        {
            if (saveFileDialog1.ShowDialog() ==
                System.Windows.Forms.DialogResult.OK)
            {
                fileName = saveFileDialog1.FileName;
                f_Out = new
                    StreamWriter(saveFileDialog1.FileName);
                f_Out.WriteLine(textBox1.Text);
                f_Out.Close();
            }
        }
    }

    private void taskToolStripMenuItem_Click(object sender, EventArgs e)
    {
        string s1,s2;

```

```

        textBox2.Clear();
        for (int i = 0; i < textBox1.Lines.Count(); i++)
        {
            s1=textBox1.Lines[i];
            s2 = i.ToString()+" ;"+s1+" ;"+s1.Length;
            if (textBox2.Text.Length > 0)
            {
                textBox2.AppendText(Environment.NewLine);
            }
            textBox2.AppendText(s2);
        }
    }

    private void условиеЗадачиToolStripMenuItem_Click(object sender,
        EventArgs e)
    {
        MessageBox.Show(" Дан текстовый файл. Переписать его
        содержимое в другой файл\n каждая строка которого имеет следующую
        структуру:\n номер строки; строка; длина строки.", "Условие задачи");
    }

    private void exitToolStripMenuItem_Click(object sender, EventArgs e)
    {
        Close();
    }
}
}

```

Результат выполнения программы представлен на рисунке 7

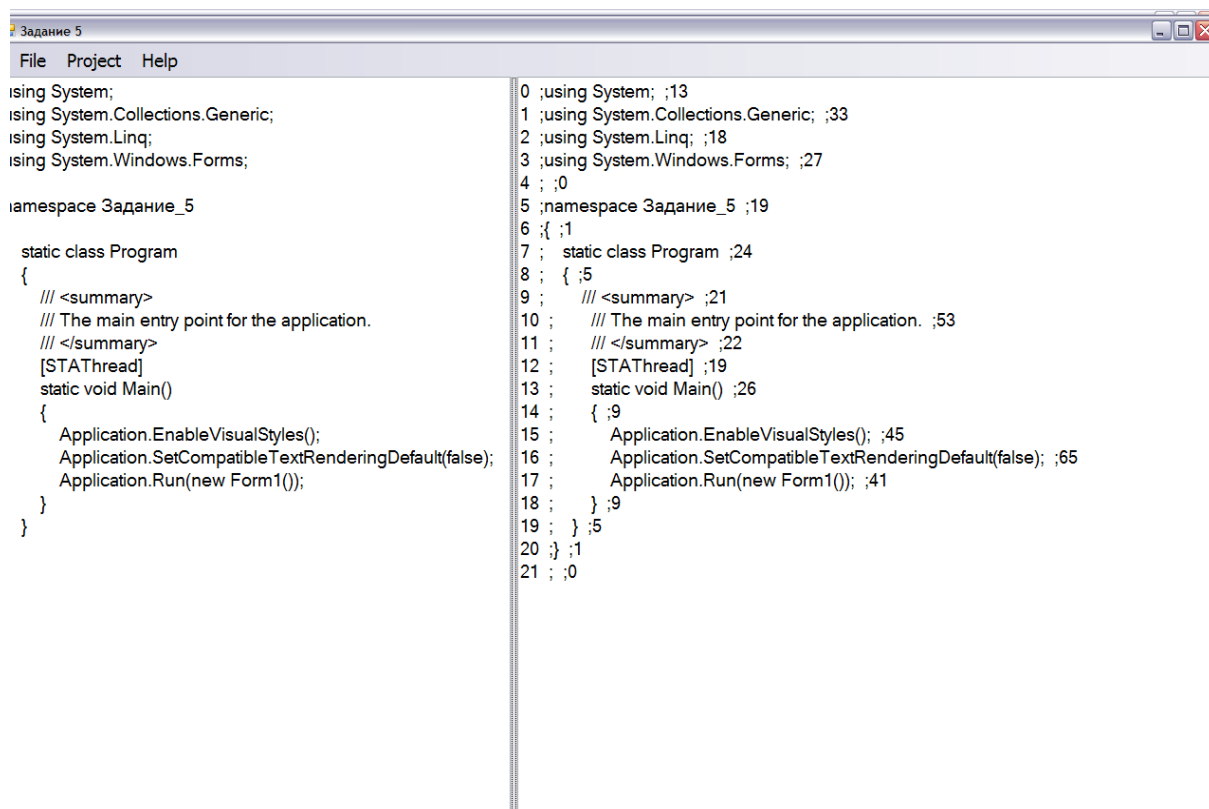


Рис.7. Результат выполнения программы.

Задание 6

Требования.

- Реализовать как Windows Application.
- Размер матрицы вводится с формы.
- Значения матрицы задаются в DataGridView.
- Преобразованная матрица выводится в DataGridView результата.
- Работа с матрицей должна быть реализована в собственном классе.

Компоненты графического интерфейса используются только для отображения, а обработка выполняется с использованием внутренних структур данных.

Задана целочисленная квадратная матрица $n * n$.

1. Удалить симметричные строки.
2. Удалить строки, в которых все значения четные.
3. Удалить все четные строки.
4. Удалить строки, содержащие заданное значение.
5. Удвоить строки, содержащие заданное значение.
6. Удалить строки, сумма элементов которых максимальна.
7. Удалить строки, образующие арифметическую прогрессию.
8. Добавить в конец строку, содержащую максимумы всех строк.
9. После строк, элементы которых строго возрастают, вставить строки, содержащие те же элементы в обратном порядке.
10. Удалить строки, содержащие все равные элементы.
11. Удалить строку с заданным номером.
12. Удалить симметричные столбцы.
13. Удалить столбцы, в которых все значения четные.
14. Удалить все четные столбцы.
15. Удалить столбцы, содержащие заданное значение.
16. Удвоить столбцы, содержащие заданное значение.
17. Удалить столбцы, сумма элементов которых максимальна.
18. Удалить столбцы, образующие арифметическую прогрессию.
19. Добавить в конец столбец, содержащий максимумы всех столбцов.
20. После столбцов, элементы которых строго возрастают, вставить столбцы, содержащие те же элементы в обратном порядке.
21. Удалить столбцы, содержащие все равные элементы.
22. Удалить столбец с заданным номером.

Пример

Удалить из матрицы строки, содержащие нули.

модуль Form1.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace Task_6
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void txtBoxN_TextChanged(object sender, EventArgs e)
        {
            //установка размера матрицы и DataGridView
            int i;
            N = Int32.Parse(txtBoxN.Text);
            DataTable matr = new DataTable("matr");
            DataColumn[] cols = new DataColumn[N];
            for (i = 0; i < N; i++)
            {
                cols[i] = new DataColumn(i.ToString());
                matr.Columns.Add(cols[i]);
            }
            for ( i = 0; i < N; i++)
            {
                DataRow newRow;

```

```

        newRow = matr.NewRow();
        matr.Rows.Add(newRow);
    }
    datGrdViewIn.DataSource = matr;
    for (i = 0; i < N; i++)
        datGrdViewIn.Columns[i].Width = 50;
    }
    // обработка матрицы
    private void btnStart_Click(object sender, EventArgs e)
    {
        datGrdViewRes.Visible = false;
        MatrMake mt = new MatrMake(N);
        mt.GridToMatrix(datGrdViewIn);
        if (mt.DelStr())
            MessageBox.Show("Все строки содержат нули");
        else
        {
            datGrdViewRes.Visible = true;
            mt.MatrixToGrid(datGrdViewRes);
        }
    }
}

```

модуль MatrMake.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Data;

```

```

namespace Task_6
{
    class MatrMake
    {
        int n_str,      //количество строк
            n_col;      //количество столбцов
        int[,] matrix; // обрабатываемая матрица

        public MatrMake(int n)
        {
            n_str = n;    // на входе матрица квадратная
            n_col = n;
            matrix = new int[n, n];
        }
        //заполнение матрицы из DataGridView
        public void GridToMatrix(DataGridView dgv)
        {
            DataGridViewCell txtCell;
            for(int i=0;i<n_str;i++)
            {
                for(int j=0; j<n_col;j++)
                {
                    txtCell = dgv.Rows[i].Cells[j];
                    string s = txtCell.Value.ToString();
                    if (s == "")
                        matrix[i, j] = 0;
                    else
                        matrix[i, j] = Int32.Parse(s);
                }
            }
        }
        //вывод матрицы в DataGridView
        public void MatrixToGrid(DataGridView dgv)
    }
}

```



```

{
    //установка размеров
    int i;
    DataTable matr = new DataTable("matr");
    DataColumn[] cols = new DataColumn[n_col];
    for (i = 0; i < n_col; i++)
    {
        cols[i] = new DataColumn(i.ToString());
        matr.Columns.Add(cols[i]);
    }
    for (i = 0; i < n_str; i++)
    {
        DataRow newRow;
        newRow = matr.NewRow();
        matr.Rows.Add(newRow);
    }
    dgv.DataSource = matr;
    for (i = 0; i < n_col; i++)
        dgv.Columns[i].Width = 50;
    // занесение значений
    DataGridViewCell txtCell;
    for ( i = 0; i < n_str; i++)
    {
        for (int j = 0; j < n_col; j++)
        {
            txtCell=dgv.Rows[i].Cells[j];
            txtCell.Value= matrix[i, j].ToString() ;
        }
    }
}

// удаление строк, содержащих нули
public bool DelStr()//true, если результат пустая матрица
{

```

```

    int i, j;
    bool ok;
    i = 0;
    while ( i < n_str)
    {
        ok = true;
        for (j = 0; j < n_col && ok; j++)
            if (matrix[i, j] == 0)
                ok = false;
        if (!ok)
        {
            //удалить i строку
            for (int k = i; k < n_str - 1; k++)
                for (j = 0; j < n_col; j++)
                    matrix[k, j] = matrix[k + 1, j];
            n_str--;
        }
        else i++;
    }
    if (n_str == 0)
        return true;
    else
        return false;
}
}
}

```

Задание 7

Требования.

Структура записи файла с информацией о студентах:

- ФИО;
- номер курса;

- номер группы;
- успеваемость — 5 экзаменов в каждой из 8 сессий;
- форма обучения.

Экзамен:

- наименование предмета,
- оценка.

Создать GUI приложение, выполняющее следующие функции.

- Создать новый файл.
- Открыть существующий файл (в формате XML, двоичном или текстовом формате).
- Сохранить файл (в формате XML, двоичном или текстовом формате).
- Вывести файл.
- Добавить запись.
- Изменить запись.
- Выполнить вариант задания.

Информацию об одном студенте и методы ее обработки оформить в отдельном классе. Для сохранения и считывания XML и двоичного файла использовать стандартную сериализацию. Компоненты графического интерфейса используются только для отображения, а обработка выполняется с использованием внутренних структур данных.

Варианты заданий.

1. На заданном курсе найти студентов, у которых успеваемость (по среднему баллу за семестр) за время обучения не ухудшается.
2. Сравнить успеваемость заданных групп на заданном курсе (по среднему баллу).
3. Найти студентов, имеющих задолженности по заданному предмету.
4. Найти студентов, имеющих в своей группе наилучший средний балл.

5. Найти номера групп, в которых нет неуспевающих.
6. Найти предметы, по которым нет неуспевающих.
7. Найти средний балл по заданному предмету.
8. На заданном курсе найти лучшую по успеваемости (по среднему баллу) группу.
9. Найти предмет, по которому наилучшая успеваемость (по среднему баллу).
10. Найти номера групп на заданном курсе, в которых есть отличники.
11. Найти курс, на котором наибольшее число студентов.
12. Вывести список предметов по курсам.
13. Вывести список студентов, имеющих более k задолженностей (k вводится).
14. На каждом курсе вывести средние баллы каждой группы.
15. Найти курс с наибольшим количеством различных предметов.
16. Найти номер курса, с наименьшим числом неуспевающих студентов.
17. Найти предметы, которые читаются на нескольких курсах.
18. На заданном курсе сравнить успеваемость студентов бюджетной и договорной форм обучения (по среднему баллу).
19. По заданному предмету сравнить успеваемость студентов бюджетной и договорной форм обучения (по среднему баллу).
20. Найти количество отличников среди студентов бюджетной и договорной форм обучения.
21. Найти группы, в которых обучаются только студенты бюджетной формы обучения.
22. По заданному предмету найти группу с наилучшей успеваемостью (по среднему баллу).
23. На каждом курсе найти количество студентов бюджетной и договорной форм обучения без задолженностей.

Список литературы

1. Троелсен Э. С# и платформа .NET 3.0 / Э.Троелсен. – СПб. : Питер, 2008 .— 1456 с.
2. Шилд Г. Полный справочник по С# / Г. Шилд . – М. : Издательский дом «Вильямс», 2004 . —752 с.
3. Рихтер Дж. CLR via С#. Программирование на платформе Microsoft .NET Framework 2.0 на языке С# / Дж. Рихтер . – М.: Издательство «Русская редакция»; СПб.: Питер, 2007.
4. Павловская Т. А. С#. Программирование на языке высокого уровня / Т.А. Павловская .— СПб : Питер, 2014 .— 432 с.
5. Тюкачев Н. А. С#. Программирование : учебное пособие : [в 3 ч.] / Н.А. Тюкачев, В.Г. Хлебостроев ; Воронеж. гос. ун-т .— Воронеж : Издательско-полиграфический центр Воронежского государственного университета, 2013

У ч е б н о е и з д а н и е

Вошинская Гильда Эдгаровна,
Лещенко Елена Михайловна

СТРУКТУРЫ И АЛГОРИТМЫ ОБРАБОТКИ ДАННЫХ

Часть 1

Учебно-методическое пособие

Издано в авторской редакции

Подписано в печать 23.04.2019. Формат 60×84/16
Уч.-изд. л. 2,6. Усл. печ. л. 3,1. Тираж 25. Заказ 200

Издательский дом ВГУ
394018 Воронеж, пл. им. Ленина, 10

Отпечатано с готового оригинал-макета
в типографии Издательского дома ВГУ
394018 Воронеж, ул. Пушкинская, 3